

Iterator in Java

Java Collections Framework — theory notes

1. Overview

An **Iterator** in Java is one of the most commonly used cursors in the Java Collections Framework. It is used to traverse (iterate through) the elements of a collection one by one.

- Traverses elements in the forward direction only.
- Removes elements safely during traversal using **remove()**.
- A universal cursor that applies to most collection types such as List, Set, and Queue.
- Although Map is part of the Java Collections Framework, it is **not** a subtype of the Collection interface. However, iterators can still be used with Map through **keySet()**, **values()**, or **entrySet()**.

2. Declaration of Iterator

```
public interface Iterator<E>
```

Here, **E** represents the type of elements to be iterated over.

3. Creating an Iterator Object

An Iterator object is created by calling the **iterator()** method on a collection object. For example, to traverse and print each element in an ArrayList:

```
Collection<String> names = new ArrayList<>();  
Iterator<String> itr = names.iterator();
```

The typical traversal pattern is a **while** loop driven by **hasNext()** and **next()**:

```
while (itr.hasNext()) {  
    String n = itr.next();  
    System.out.println(n);  
}
```

hasNext() checks whether an element remains ahead of the cursor, and **next()** both retrieves that element and advances the cursor past it. Running this pattern over an ArrayList containing "A", "B", "C" prints each element in insertion order: A, B, C.

4. Hierarchy of Iterator

- Iterator is part of the **java.util** package and is used by collection classes to traverse elements.
- Collection classes implement the **Iterable** interface, which provides the **iterator()** method that returns an Iterator object.
- **ListIterator** extends Iterator, adding bi-directional traversal capability.

5. Methods of the Iterator Interface

The Iterator interface defines three methods:

Method	Description
<code>hasNext()</code>	Returns true if the iteration has more elements.
<code>next()</code>	Returns the next element in the iteration. Throws NoSuchElementException if no more elements are present.
<code>remove()</code>	Removes the last element returned by <code>next()</code> . Can be called only once per call to <code>next()</code> ; calling it again without an intervening <code>next()</code> throws IllegalStateException .

```
Iterator<Integer> it = list.iterator();
it.next();
it.remove(); // Works fine
```

Note: `remove()` can throw two exceptions:

- **UnsupportedOperationException** — if the remove operation is not supported by this iterator.
- **IllegalStateException** — if `remove()` is called before `next()` or more than once per `next()` call.

6. Fail-Fast Behavior and Why Iterator Matters

Most Iterator implementations in the standard collections (`ArrayList`, `HashSet`, `HashMap`'s views, etc.) are **fail-fast**: if the underlying collection is structurally modified after the iterator is created — by any means other than the iterator's own `remove()` (or `add()`, for `ListIterator`) — the iterator detects this on its next operation and throws a **ConcurrentModificationException**. This is a best-effort safety check, not a guarantee, but it exists specifically to catch unsafe modification during traversal early rather than let the program silently produce inconsistent results.

This is precisely why the Iterator's own `remove()` method exists: it is the only structural modification the iterator is aware is happening, so it can update its internal bookkeeping (such as the expected modification count) to stay consistent, instead of tripping its own fail-fast check.

The enhanced for-loop (`for (String s : list)`) is syntactic sugar that uses an Iterator internally, but it does not expose that iterator to the programmer — so `remove()` cannot be called from inside a for-each loop. Whenever safe removal, or any need to inspect `hasNext()` before consuming an element, is required, an explicit Iterator (obtained via `collection.iterator()`) must be used instead of the for-each loop.

7. Internal Working (Step by Step)

Consider a LinkedList/List object named **cities**. The internal cursor of an Iterator moves as follows:

Step 1 — Create the iterator

```
Iterator<String> citiesIterator = cities.iterator();
```

The cursor is positioned *before* the first element of the list.

Step 2 — First call to next()

```
citiesIterator.hasNext();  
citiesIterator.next();
```

The cursor now points to the first element in the list.

Step 3 — Second call to next()

```
citiesIterator.hasNext();  
citiesIterator.next();
```

The cursor now points to the second element in the list.

Step 4 — Continue to the end

Repeating this process moves the cursor to the last element of the list.

Step 5 — After the final element

```
citiesIterator.hasNext();
```

Once the cursor is past the final element, hasNext() returns **false**.

Note: Java's Iterator supports only forward-direction traversal, so it is described as a *uni-directional* cursor. In contrast, **ListIterator** is bi-directional, allowing traversal in both directions.

8. Removing Elements During Traversal

One of the main reasons to prefer an Iterator over a plain for-each loop is that it allows elements to be removed **safely** while the collection is being traversed. Modifying a collection directly with a for-each loop (or by calling the collection's own remove() method mid-iteration) throws a **ConcurrentModificationException**. The Iterator's own remove() method avoids this because it updates the iterator's internal state in step with the removal.

The core pattern is: fetch an element with next(), inspect it, and call remove() on the iterator itself if it should be dropped:

```
int i = itr.next();  
if (i % 2 != 0) {  
    itr.remove(); // removes the element just returned by next()  
}
```

Applied to a list of integers 0–9, this pattern removes every odd number and leaves only the even ones (0, 2, 4, 6, 8) behind — the list is mutated in place, in a single forward pass, without ever throwing ConcurrentModificationException.

9. Quick Summary

Aspect	Detail
Package	java.util
Direction	Forward only (uni-directional)
Created via	collection.iterator()
Core methods	hasNext(), next(), remove()
Safe removal	Yes, via remove() during traversal
Works with Map?	Indirectly — via keySet(), values(), entrySet()
Bi-directional alternative	ListIterator
Modification safety	Fail-fast — throws ConcurrentModificationException on unsafe external modification